

JUNIT 5 & MOCKITO

Advanced Interview Preparation Guide for Java/Spring Boot Developers

Study Scope: This comprehensive roadmap covers the core functionality, advanced techniques, and common interview questions regarding JUnit and Mockito for enterprise Java roles.

1. The Fundamentals of Unit Testing

Definition: Unit Testing is the process of testing one small unit of code independently. A unit can be a Method, Function, Service Class, or Utility Class.

```
// Example Method
public int add(int a, int b) {
    return a + b;
}
```

Instead of running the whole application, we test only this specific method in isolation.

Why is Unit Testing Essential?

WITHOUT TESTING	WITH UNIT TESTING
Write code → Run entire application → Check manually → <i>Time consuming</i>	Write code → Run JUnit Test → Framework checks result → <i>Fast</i>

Benefits include: finding bugs early, preventing future bugs, improving code quality, enabling safe refactoring, and faster development.

2. Categories of Testing

TESTING TYPE	PURPOSE
Unit Testing	Tests one class or method (e.g., Test only Service)
Integration Testing	Tests communication between classes/modules (e.g., Service + Repository + Database)
System Testing	Tests the entire application
Acceptance Testing	Tests business requirements

3. JUnit Framework Overview

JUnit is the most popular Java testing framework, used to write and execute test cases, compare expected versus actual results, and generate test reports.

```
<!-- Maven Dependency -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

This starter dependency includes JUnit 5, Mockito, AssertJ, and Hamcrest.

4. The JUnit 5 Lifecycle

Understanding the execution order of lifecycle annotations is critical:

- `@BeforeAll` : Runs only once before all test methods. Useful for creating a heavy Database Connection. Must be static.
- `@BeforeEach` : Runs before every single test. Useful for initializing fresh objects for each test.

- `@Test` : Marks a method as a test method. Without this, JUnit ignores the method.
- `@AfterEach` : Runs after every test. Commonly used for cleanup operations.
- `@AfterAll` : Runs once after all tests complete. Used to close resources.

5. Assertions Explained

Assertions compare the **Expected** value against the **Actual** value. Here are the detailed explanations for the core assertion methods:

- `assertEquals(10, result)` : Evaluates if the expected value strictly matches the actual output. (e.g., Pass if Expected = 10, Actual = 10).
- `assertNotEquals(20, result)` : Ensures that the actual result is intentionally different from the provided expected value (e.g., checking that a generated ID is not 0).
- `assertTrue(age > 18)` : Validates that a supplied boolean condition evaluates to true. If it evaluates to false, the test fails.
- `assertFalse(list.isEmpty())` : Validates that a supplied boolean condition evaluates to false.
- `assertNull(employee)` : Confirms that the target object reference points to a null value. Useful for testing failure scenarios or missing data.
- `assertNotNull(employee)` : Confirms that the target object has been initialized and is not null. Commonly used to check if an object was successfully created.
- `assertThrows(RuntimeException.class, () -> service.getEmployee(1L))` : Tests that the block of code inside the executable explicitly throws the expected Exception type. If no exception is thrown, the test fails.
- `assertDoesNotThrow(() -> service.save(employee))` : Verifies that the provided code block executes cleanly without throwing any exceptions at all.

6. Introduction to Mockito

Mockito creates fake objects to isolate the class under test. For example, a unit test should not connect to a real MySQL database; instead, Mockito replaces the Repository with a mock.

Real Object: Repository → Database

Mock Object: Repository → Returns Fake Data

7. Core Mockito Annotations

- `@Mock` : Creates a completely fake dependency (e.g., a Fake Repository).
- `@InjectMocks` : Creates the actual class under test and injects the mocked dependencies into it.
- `@ExtendWith(MockitoExtension.class)` : The preferred approach in JUnit 5 to initialize mocks at the class level.

8. Defining Mock Behavior (Stubbing)

Stubbing tells the mock how to respond when its methods are invoked.

- `when(...).thenReturn(...)` : Defines how the mock should behave and what fake object it should return. Example:

```
when(repository.findById(1L)).thenReturn(Optional.of(employee));
```
- `thenThrow(...)` : Simulates an exception thrown by the mocked dependency. Example:

```
when(repository.findById(1L)).thenThrow(new RuntimeException());
```

9. Verification Methods Explained

Verification checks method invocation to confirm if a specific method was called on the mock. Here are the specific verify methods detailed:

- `verify(repository).save(employee)` : The default verification. Checks if the `save` method was called exactly one time with the `employee` object. "Was save() called? Yes → Pass."
- `verify(repository, times(2)).save(employee)` : Verifies that the method was called exactly two times during the test execution.
- `verify(repository, never()).delete(employee)` : Ensures that the `delete` method was completely ignored and never called during the execution flow.
- `verify(repository, atLeast(1)).save(employee)` : Confirms that the method was invoked a minimum of 1 time (but could be more).
- `verify(repository, atMost(2)).save(employee)` : Confirms that the method was invoked no more than 2 times.

10. Argument Matchers & Captors

Matchers:

- `any()` : Matches any object of a given type. Example:
`when(repository.save(any(Employee.class)))` .
- `eq()` : Exact value matcher. Example: `verify(repository).findById(eq(1L));` .

ArgumentCaptor:

Captures arguments passed to a mocked method, which is highly useful for checking object values right before they are saved.

```
ArgumentCaptor<Employee> captor = ArgumentCaptor.forClass(Employee.class);
verify(repository).save(captor.capture());
Employee emp = captor.getValue(); // Inspect 'emp' fields
```

11. Mock vs. Spy

FEATURE	MOCK	SPY
Authenticity	Completely fake	Partially fake (Real object base)
Execution	No real methods run	Real methods execute unless explicitly mocked
Primary Use Case	Isolating external dependencies	Partial mocking of an object

12. Complete Spring Boot Test Example

```
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {

    @Mock
    EmployeeRepository repository;

    @InjectMocks
    EmployeeService service;

    @Test
    void testGetEmployee() {
        Employee emp = new Employee();
        emp.setId(1L);
        emp.setName("John");

        // 1. Mock Behavior
        when(repository.findById(1L)).thenReturn(Optional.of(emp));

        // 2. Call Service
        Employee result = service.getEmployee(1L);

        // 3. Assert Result
        assertEquals("John", result.getName());

        // 4. Verify Invocation
        verify(repository).findById(1L);
    }
}
```

13. Best Practices

- Test one method at a time.
- One test should verify one behavior.
- Use meaningful test method names (e.g., `testGetEmployeeById()` instead of `test1()`).
- Never connect to a real database in unit tests.
- Mock external services (REST APIs, databases, Kafka, etc.).

14. Essential Interview Questions & Answers

Q1. What is Unit Testing?

Testing an individual method or class in isolation to ensure it behaves as expected.

Q2. What is the difference between JUnit and Mockito?

JUnit is the testing framework that executes tests and provides assertions. Mockito is the mocking framework that creates mock objects and stubs/verifies method calls.

Q3. What is Mocking?

Replacing real dependencies with fake objects to isolate the class under test.

Q4. Why use Mockito?

To avoid calling real databases, REST APIs, file systems, or other external dependencies during unit tests.

Q5. What is the difference between `@Mock` and `@InjectMocks`?

`@Mock` creates a fake dependency. `@InjectMocks` creates the real class under test and injects the mocked dependencies into it.

Q6. Why use `verify()`?

To ensure that expected interactions with dependencies actually occurred (e.g., checking that `repository.save(employee)` was called exactly once).

Q7. What is the difference between `when().thenReturn()` and `verify()`?

`when().thenReturn()` defines *how* a mock should behave, whereas `verify()` checks *whether* a method on the mock was actually called.

Q8. Why don't we use a real database in unit testing?

Because unit tests should be Fast, Independent, Repeatable, and Focused only on the business logic of the class being tested. Using a real database is more appropriate for integration testing.

Q9. What is the difference between @Mock and @MockBean?

`@Mock` is a standard Mockito annotation used in lightweight, isolated unit tests. `@MockBean` is a Spring Boot annotation used to add or replace a bean with a mock inside the full Spring ApplicationContext during integration testing.

Q10. How do you mock a method that returns void?

Because void methods return nothing, you cannot use `when().thenReturn()`. Instead, you use `doNothing()`, `doThrow()`, or `doAnswer()`. For example: `doThrow(new RuntimeException()).when(mock).voidMethod();`